Nantes Université

POLYTECH NANTES

ETN 5 – Polytech Nantes

Langages de description matérielle et réutilisation

Rapport de mini-projet

# Réutilisation de codes HDL et HDL réutilisable

WU Riluke & LIU Chengpeng

Enseignant : Sébastien Pillement

Octobre 2026

# Table des matières

**Introduction** **3**

**2 Récupération d'un modèle de RAM** **3**

**3 Réutilisation du bloc Sync_Ram** **3**

**4 Prise en main d'un modèle de processeur** **3**

4.1 Valider le processeur RISC-V . . . . . 3

4.2 Manipuler les instructions RISC-V . . . . . 4

4.3 Adapter et implémenter les instructions RISC-V . . . . . 5

4.3.1 Modes d'adressage . . . . . 5

4.3.2 Corrections du modèle fourni . . . . . 5

4.3.3 Du jeu RV32I au jeu RV32IM . . . . . 7

4.3.4 Implémentation et validation de l'extension M . . . . . 8

4.4 Adapter la µ-architecture du processeur RISC-V . . . . . 10

**5 Création du système** **12**

5.1 Accès aux données dans la mémoire externe . . . . . 12

5.2 Synchronisation avec le bloc quadram . . . . . 13

5.3 Micro-architecture du processeur . . . . . 14

5.4 Bloc système en VHDL . . . . . 14

5.5 Validation . . . . . 16

5.6 Différences avec le premier programme . . . . . 18

**Conclusion** **18**

**Table des figures** **19**

**Liste des tableaux** **19**

**Bibliographie** **19**

# Introduction

**À compléter :** objectifs du mini-projet (réutilisation de blocs VHDL, généricité, intégration d'un processeur RISC-V écrit en Verilog), outils utilisés (ModelSim SE-64 2020.4) et plan du rapport [1].

# 2 Récupération d'un modèle de RAM

**À compléter :** analyse du modèle `ramchip.vhd` et de son testbench ; passage au type `unsigned` ; généricité (`Nbits_addr`, `Nbits_data`) ; signal CS asynchrone (sortie haute impédance, écriture autorisée seulement si `cs = '1'`) ; simulations (Bench et Bench2).

# 3 Réutilisation du bloc Sync_Ram

**À compléter :** décodeur d'adresses et sa simulation ; bloc `quadram` (4 instances de `Sync_Ram` et le décodeur) ; validation ; version générique avec `for ... generate` (`Nbits_sel`, `Nbits_addr`, `Nbits_data`).

# 4 Prise en main d'un modèle de processeur

Le processeur fourni sur Madoc (`riscvproc-2.v`) est un modèle Verilog monocycle qui implémente une partie du jeu d'instructions RV32I [2] : `add`, `sub`, `sll`, `srl`, `addi`, `andi`, `ori`, `lw`, `sw` et `ebreak`. Il contient une mémoire interne `memory` de 4096 mots de 32 bits, qui sert à la fois au programme et aux données, ainsi que les 32 registres `x0` à `x31`. Une instruction est exécutée toutes les `Step` = 10 ns ; le module `system` n'a aucun port, c'est un modèle purement simulable.

Le fichier utilise des constructions SystemVerilog (`localparam int unsigned`) : nous l'avons renommé avec l'extension `.sv` et compilé avec `vlog -sv`. L'instruction `ebreak` termine la simulation par `$finish` ; nous lançons donc `vsim` avec l'option `-onfinish stop`, qui arrête la simulation sans fermer le design et permet d'examiner ensuite les registres et la mémoire.

## 4.1 Valider le processeur RISC-V

Pour faire exécuter un programme, la mémoire du processeur est initialisée au début de la simulation à partir d'un fichier texte, avec la tâche système `$readmemh` placée dans le bloc `initial` du processeur (code 1). Le fichier contient un mot de 32 bits en hexadécimal par ligne ; `@0` donne l'adresse du premier mot et `//` introduit un commentaire. Le chemin est relatif au répertoire courant de ModelSim : si le fichier n'est pas trouvé, ModelSim l'indique par un avertissement, la mémoire reste à `X` et le processeur affiche « erreur : mauvaise valeur dans op-code » à chaque instruction.

Code 1 – Chargement du programme dans la mémoire du processeur

```
initial begin
  $readmemh("instr2.txt", memory);
  // initialize program counter to 0
  pc = 0;
  ...
end
```

Le tableau 1 décode le programme `instr2.txt` fourni sur Madoc. Avec le modèle d'origine, les registres sont corrects à la fin du programme (x1 = 1, x2 = 6), mais l'instruction `sw` n'écrit pas au mot 65 : elle écrit au mot 0 et écrase la première instruction du programme (`memory[0]` vaut 6 et `memory[65]` reste à `X`). C'est cette erreur qui nous a conduits aux corrections de la section 4.3.2.

| Adresse | Code | Instruction | Effet |
|---|---|---|---|
| 0 | `00200093` | `addi x1, x0, 2` | x1 = 2 |
| 1 | `00400113` | `addi x2, x0, 4` | x2 = 4 |
| 2 | `00110133` | `add x2, x2, x1` | x2 = 6 |
| 3 | `001150b3` | `srl x1, x2, x1` | x1 = 6 décalé de 2 bits = 1 |
| 4 | `0420a023` | `sw x2, 64(x1)` | le mot 64 + x1 = 65 reçoit 6 |
| 5 | `00100073` | `ebreak` | fin de la simulation |

TABLE 1 – Programme `instr2.txt`

## 4.2 Manipuler les instructions RISC-V

Toutes les instructions RISC-V font 32 bits. Le modèle utilise trois formats (tableau 2) : le format R pour les opérations entre registres, le format I pour les opérations avec un immédiat de 12 bits et pour `lw`, et le format S pour `sw`, dont l'immédiat est découpé en deux champs afin que `rs1` et `rs2` restent à la même place que dans le format R.

| | 31 – 25 | 24 – 20 | 19 – 15 | 14 – 12 | 11 – 7 | 6 – 0 |
|---|---|---|---|---|---|---|
| R | funct7 | rs2 | rs1 | funct3 | rd | opcode |
| I | `imm[11:0]` | | rs1 | funct3 | rd | opcode |
| S | `imm[11:5]` | rs2 | rs1 | funct3 | `imm[4:0]` | opcode |

TABLE 2 – Formats d'instructions utilisés (spécification RISC-V, chapitre 2.2)

L'instruction `add x9, x20, x21` est de format R. Ses champs sont donnés dans le tableau 3 : le code obtenu est `0000 0001 0101 1010 0000 0100 1011 0011`, soit `0x015A04B3`. L'opcode `0110011` désigne les opérations entre registres ; `funct3` et `funct7` choisissent l'opération (`funct7` vaut `0100000` pour `sub`).

| funct7 | rs2 = x21 | rs1 = x20 | funct3 | rd = x9 | opcode |
|---|---|---|---|---|---|
| 0000000 | 10101 | 10100 | 000 | 01001 | 0110011 |

TABLE 3 – Codage de `add x9, x20, x21`

## 4.3 Adapter et implémenter les instructions RISC-V

### 4.3.1 Modes d'adressage

Le mode d'adressage indique où se trouve l'opérande d'une instruction (tableau 4).

| Mode | Principe | Exemples |
|---|---|---|
| Immédiat | L'opérande est une constante codée dans l'instruction elle-même. | `addi x1, x0, -7` |
| Direct | L'instruction désigne directement l'emplacement de l'opérande : un registre, ou une adresse mémoire fixe. | `add x9, x20, x21`<br>`lw x1, 64(x0)` |
| Indirect | Un registre contient l'adresse de l'opérande en mémoire (pointeur). | `lw x1, 0(x2)`<br>`sw x3, 4(x2)` |

TABLE 4 – Modes d'adressage

En RISC-V, `lw` et `sw` n'ont qu'un seul mode : l'adresse est la somme d'un registre de base `rs1` et d'un déplacement signé de 12 bits, ce qui s'écrit `offset(rs1)`. L'adressage direct en est le cas particulier `rs1` = `x0` : `x0` vaut toujours 0, donc l'adresse 64 de `lw x1, 64(x0)` est entièrement codée dans l'instruction. L'adressage indirect en est le cas `offset` = 0. Dans ce modèle, la mémoire est adressée par mots de 32 bits (`memory[addr]`) et non par octets comme dans la spécification : `4(x2)` désigne le mot situé 4 mots après l'adresse contenue dans `x2`.

### 4.3.2 Corrections du modèle fourni

Pour tester les immédiats négatifs, les accès mémoire avec un déplacement positif et négatif, et le registre `x0`, nous avons écrit le programme `test_base` (code 2). Ses résultats sont donnés dans le tableau 5, relevés dans ModelSim avec la commande `examine` (code 3) : le modèle d'origine présente trois erreurs.

Code 2 – Programme de test `test_base.s`

```
addi x1, x0, -7      # x1 = -7
addi x2, x0, 100     # x2 = 100
sub  x3, x2, x1      # x3 = 107
andi x4, x1, -16     # x4 = -16
ori  x5, x0, -1      # x5 = -1
addi x6, x0, 3       # x6 = 3
sll  x7, x2, x6      # x7 = 800
```

```
srl  x8, x5, x6      # x8 = 0x1FFFFFFF = 536870911
sw   x3, 4(x2)       # mot 104 = 107
sw   x1, -4(x2)      # mot 96 = -7
lw   x9, 4(x2)       # x9 = 107
lw   x10, -4(x2)     # x10 = -7
addi x0, x0, 5       # x0 doit rester a 0
add  x11, x0, x0     # x11 = 0
ebreak
```

| | Instruction | Attendu | Modèle d'origine | Après correction |
|---|---|---|---|---|
| x0 | `addi x0, x0, 5` | 0 | 5 | 0 |
| x1 | `addi x1, x0, -7` | −7 | 4089 | −7 |
| x3 | `sub x3, x2, x1` | 107 | −3989 | 107 |
| x4 | `andi x4, x1, -16` | −16 | 4080 | −16 |
| x5 | `ori x5, x0, -1` | −1 | 4095 | −1 |
| x8 | `srl x8, x5, x6` | 536870911 | 511 | 536870911 |
| x9 | `lw x9, 4(x2)` | 107 | 4089 | 107 |
| x10 | `lw x10, -4(x2)` | −7 | 4089 | −7 |
| x11 | `add x11, x0, x0` | 0 | 10 | 0 |
| mot 104 | `sw x3, 4(x2)` | 107 | X | 107 |
| mot 96 | `sw x1, -4(x2)` | −7 | X | −7 |
| mot 0 | première instruction | `ff900093` | 4089 | `ff900093` |

TABLE 5 – Résultats de `test_base` (x2 = 100, x6 = 3 et x7 = 800 sont corrects dans les deux cas)

Code 3 – Transcript ModelSim : `test_base` avec le modèle d'origine, puis avec le modèle corrigé

```
# Stopped at riscvproc_original.sv line 149
examine -radix decimal /system/regs
# {5 4089 100 -3989 4080 4095 3 800 511 4089 4089 10 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0}
examine -radix decimal {/system/memory[0]} {/system/memory[96]} {/system/memory
   [104]}
# 4089 x x

# Stopped at riscvproc-2.sv line 149
examine -radix decimal /system/regs
# {0 -7 100 107 -16 -1 3 800 536870911 107 -7 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
   0 0}
examine -radix hexadecimal {/system/memory[0]}
# ff900093
examine -radix decimal {/system/memory[96]} {/system/memory[104]}
```

```
# -7 107
```

**Immédiats non étendus en signe.** Le modèle définissait `` `define I_IMM $signed(instr[31:20]) ``. Or, en Verilog, dès qu'un opérande d'une expression est non signé (ici le registre `regs[rs1]`), toute l'expression est évaluée en non signé et l'immédiat de 12 bits est complété par des zéros : $-7$ (`0xFF9`) devient 4089. C'est l'erreur sur x1, x3, x4 et x5. Nous faisons l'extension de signe explicitement, en recopiant le bit 31 par concaténation, pour le format I comme pour le format S (code 4).

**Adresse des accès mémoire.** L'adresse était calculée par `addr = `LOAD_OFFSET + regs[`RS1];` avec `` `define LOAD_OFFSET instr[31:20] << 12 ``. Une macro est une simple substitution de texte : la ligne devient `addr = instr[31:20] << 12 + regs[`RS1];`, et comme + est prioritaire sur <<, le déplacement est décalé de 12 + `rs1` bits. Dans nos programmes, l'adresse de 12 bits valait donc toujours 0 : les `lw` lisaient le mot 0 et les `sw` écrasaient la première instruction du programme (d'où x9 = x10 = 4089 et le mot 0 égal à 4089). Le décalage de 12 bits est de toute façon faux : il correspond au format de `lui`, pas aux accès mémoire. L'adresse vaut maintenant `rs1` + immédiat I pour `lw`, et `rs1` + immédiat S pour `sw`.

**Registre x0.** `addi x0, x0, 5` modifiait `x0`, qui doit toujours valoir 0 (spécification, chapitre 2.1). Nous remettons `regs[0]` à 0 à la fin de chaque instruction.

Code 4 – Corrections du modèle (immédiats, adresses, x0)

```
`define I_IMM   {{20{instr[31]}}, instr[31:20]}              // I-type immediate
`define S_IMM   {{20{instr[31]}}, instr[31:25], instr[11:7]}  // S-type immediate
...
`LW: begin
  addr      = regs[`RS1] + `I_IMM;
  regs[`RD] = memory[addr];
end
...
`SW: begin
  addr         = regs[`RS1] + `S_IMM;
  memory[addr] = regs[`RS2];
end
...
regs[0] = 0;   // x0 is hardwired to 0
```

### 4.3.3 Du jeu RV32I au jeu RV32IM

L'extension M ajoute huit instructions de multiplication et de division entières (tableau 6). Ce sont toutes des instructions de format R, avec le même opcode `0110011` que `add` ; elles se distinguent des opérations

de RV32I par `funct7` = `0000001` (au lieu de `0000000` ou `0100000`), et `funct3` choisit l'opération.

| Instruction | funct3 | Résultat |
|---|---|---|
| `MUL` | `000` | 32 bits de poids faible de `rs1` × `rs2` |
| `MULH` | `001` | 32 bits de poids fort, signé × signé |
| `MULHSU` | `010` | 32 bits de poids fort, `rs1` signé × `rs2` non signé |
| `MULHU` | `011` | 32 bits de poids fort, non signé × non signé |
| `DIV` | `100` | quotient signé, arrondi vers 0 |
| `DIVU` | `101` | quotient non signé |
| `REM` | `110` | reste signé, du signe du dividende |
| `REMU` | `111` | reste non signé |

TABLE 6 – Instructions de l'extension M (opcode `0110011`, funct7 = `0000001`)

Les modifications à apporter au processeur sont donc les suivantes :

— **Décodage** : aucun nouvel opcode n'est nécessaire. Le processeur décode déjà les opérations entre registres sur le couple {`funct3`, `funct7`} ; il suffit d'y ajouter les huit combinaisons.

— **Chemin de données** : un multiplieur $32 \times 32$ bits produisant 64 bits, et un diviseur donnant le quotient et le reste, en versions signée et non signée. Les 32 bits de poids faible du produit ne dépendent pas du signe des opérandes, d'où une seule instruction `MUL` ; les 32 bits de poids fort en dépendent, d'où les trois variantes `MULH`.

— **Cas particuliers** : RISC-V ne déclenche pas d'exception lors d'une division ; la spécification fixe le résultat de la division par zéro et du dépassement (tableau 7).

| Cas | `DIV` | `DIVU` | `REM` | `REMU` |
|---|---|---|---|---|
| Division par 0 | $-1$ | $2^{32}-1$ | dividende | dividende |
| Dépassement $-2^{31}/(-1)$ | $-2^{31}$ | – | 0 | – |

TABLE 7 – Résultats imposés par la spécification pour la division

### 4.3.4 Implémentation et validation de l'extension M

Nous avons ajouté les huit codes sous la même forme que les instructions existantes, par exemple `` `define MUL {3'b000, 7'b0000001} ``, et huit cas dans le décodage des opérations entre registres (code 5). Trois points de Verilog demandent de l'attention :

— **Largeur des expressions.** Verilog calcule une expression sur la largeur de son plus grand opérande ou de sa destination. Avec des registres de 32 bits, le produit serait tronqué à 32 bits et la partie haute perdue. Les opérandes sont donc étendus à 64 bits dans la variable `prod` : extension de signe `{{32{x[31]}}, x}` pour un opérande signé, zéros `{32'b0, x}` pour un opérande non signé.

— **Division signée.** Les deux opérandes passent par `$signed`. La division signée de Verilog arrondit vers zéro et le reste prend le signe du dividende, comme en RISC-V.

— **Cas particuliers.** En Verilog, une division par zéro donne `X` ; ces cas sont donc traités avant la division, avec `if`/`else`. Nous avons évité l'opérateur conditionnel `? :` : si l'une de ses branches est non signée (par exemple `32'hFFFFFFFF`), toute l'expression devient non signée, et la division de l'autre branche est alors calculée en non signé ($-7/2$ donne 2147483644 au lieu de $-3$).

Code 5 – Extrait de l'implémentation de l'extension M

```
reg[63:0] prod;   // 64-bit product for MULH, MULHSU, MULHU
...
`MUL:    regs[`RD] = regs[`RS1] * regs[`RS2];  // 32-bit target: low half
`MULH:   begin
           prod = {{32{regs[`RS1][31]}}, regs[`RS1]} * {{32{regs[`RS2][31]}}, regs[
   `RS2]};
           regs[`RD] = prod[63:32];
         end
...
`DIV:    if (regs[`RS2] == 0)
           regs[`RD] = 32'hFFFFFFFF;                    // division by 0: -1
         else if (regs[`RS1] == 32'h80000000 && regs[`RS2] == 32'hFFFFFFFF)
           regs[`RD] = 32'h80000000;                    // overflow: -2^31
         else
           regs[`RD] = $signed(regs[`RS1]) / $signed(regs[`RS2]);
```

Le programme `test_rv32im` teste les huit instructions et les trois cas particuliers (tableau 8 et code 6). Avant la modification, le processeur affichait onze fois « erreur : mauvaise instruction arithmétique » (une fois par instruction de l'extension M) et les registres correspondants restaient à 0 ; après la modification, toutes les valeurs sont celles attendues.

| Instruction | Registre | Résultat | Remarque |
|---|---|---|---|
| `mul x3, x1, x2` | x3 | $-14$ | |
| `div x4, x1, x2` | x4 | $-3$ | arrondi vers 0 |
| `rem x5, x1, x2` | x5 | $-1$ | signe du dividende |
| `divu x6, x1, x2` | x6 | 2147483644 | `0xFFFFFFF9` / 2 |
| `div x7, x1, x0` | x7 | $-1$ | division par 0 |
| `mulh x8, x1, x2` | x8 | $-1$ | partie haute de $-14$ |
| `mulhu x9, x1, x2` | x9 | 1 | `0xFFFFFFF9` $\times$ 2 = `0x1FFFFFF2` |
| `remu x10, x1, x2` | x10 | 1 | |
| `mulhsu x16, x1, x2` | x16 | $-1$ | |
| `div x14, x11, x12` | x14 | $-2^{31}$ | dépassement |
| `rem x15, x11, x12` | x15 | 0 | dépassement |

TABLE 8 – Résultats de `test_rv32im`, avec x1 = $-7$, x2 = 2, x11 = $-2^{31}$ et x12 = $-1$

Code 6 – Transcript ModelSim : registres après `test_rv32im`

```
# ** Note: $finish    : riscvproc-2.sv(196)
#    Time: 190 ns  Iteration: 0  Instance: /system
examine -radix decimal /system/regs
# {0 -7 2 -14 -3 -1 2147483644 -1 -1 1 1 -2147483648 -1 31 -2147483648 0 -1 0 0 0 0
   0 0 0 0 0 0 0 0 0 0 0}
```

## 4.4 Adapter la µ-architecture du processeur RISC-V

Pour pouvoir relier le processeur au bloc `quadram`, le module `system`, qui n'avait pas de port, devient le module `riscvproc` avec une horloge, un reset et un bus de données (tableau 9). L'adresse fait 12 bits, comme le bus d'adresse interne du processeur : elle désigne 4096 mots, soit les 4 RAM de 1024 mots de la partie 5.

| Port | Direction | Largeur | Rôle |
|---|---|---|---|
| `clock` | entrée | 1 | horloge |
| `reset` | entrée | 1 | remise à zéro synchrone, active à 1 |
| `address` | sortie | 12 | adresse du mot de données |
| `dataToWrite` | sortie | 32 | donnée à écrire |
| `dataToRead` | entrée | 32 | donnée lue |
| `we` | sortie | 1 | autorisation d'écriture |

TABLE 9 – Ports ajoutés au processeur

Les autres modifications sont les suivantes (code 7) :

— **Horloge.** Le paramètre `Step` et l'attente `#Step` sont remplacés par `@(negedge clock)` : le processeur exécute une instruction par période d'horloge, sur le front descendant. Le bloc `quadram` échantillonne ses entrées sur le front montant, une demi-période plus tard : les signaux du bus sont alors stables.

— **Reset synchrone**, actif à 1 : sur un front descendant avec `reset` = 1, `pc` et les registres reprennent leur valeur initiale, 0. La mémoire programme n'est pas modifiée.

— **Lecture de l'instruction par affectation bloquante.** Le modèle d'origine utilisait `instr <= memory[pc];`. Avec une affectation non bloquante, `instr` n'est mise à jour qu'après le décodage, si bien que chaque période décodait l'instruction lue à la période précédente ; à la première période, l'instruction était encore indéfinie, d'où un message « mauvaise valeur dans op-code » au démarrage. Avec `instr = memory[pc];`, l'instruction est lue et exécutée dans la même période. C'est aussi nécessaire pour la partie 5, où `lw` et `sw` attendent la mémoire alors que `instr` doit rester stable.

— **Sorties du bus** initialisées à 0 : à cette étape, les données sont encore dans la mémoire interne.

Code 7 – Boucle principale du processeur avec horloge et reset

```
always begin // one instruction per clock period
  @(negedge clock);
  if (reset) begin
    // synchronous reset: same values as at the start of the simulation
    pc = 0;
    for (index = 0; index < NRegs; index = index + 1) begin
      regs[index] = 0;
    end
  end else begin
    instr = memory[pc];   // instruction fetch (blocking: executed in this same
   period)
    pc    = pc + 1;       // increment program counter
    addr  = 0;            // address initial value
    case(`OPCODE)         // decode instruction
      ...
    endcase
    regs[0] = 0;
  end // if (reset) ... else
end
```

Le testbench `tb_riscvproc.sv` instancie le processeur, génère une horloge de période 10 ns et maintient le reset à 1 jusqu'à 12 ns, ce qui laisse un front descendant de reset à 10 ns. La première instruction est exécutée à 20 ns, puis une instruction toutes les 10 ns (figure 1 [1]). Les résultats sont identiques à ceux de la section précédente pour les trois programmes : `test_rv32im` (18 instructions) se termine à 190 ns,

1. Les chronogrammes de ce rapport sont tracés à partir des fichiers VCD enregistrés par ModelSim pendant les simulations (commandes `vcd file` et `vcd add`).

`test_base` à 160 ns et `instr2` à 70 ns.

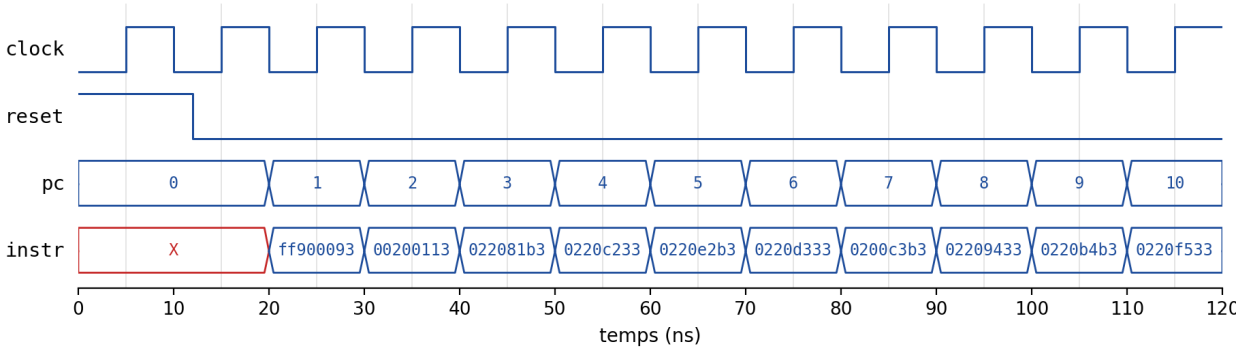



FIGURE 1 – Processeur avec horloge et reset, début de `test_rv32im` : une instruction par période

# 5 Création du système

Dans le système final (figure 2), le programme reste dans la mémoire interne du processeur et les données sont stockées dans le bloc `quadram` de la partie 3. Le bloc de plus haut niveau, `systeme`, est écrit en VHDL : il instancie le processeur Verilog en tant que « component » ainsi que le bloc `quadram`, et les relie par des signaux internes.

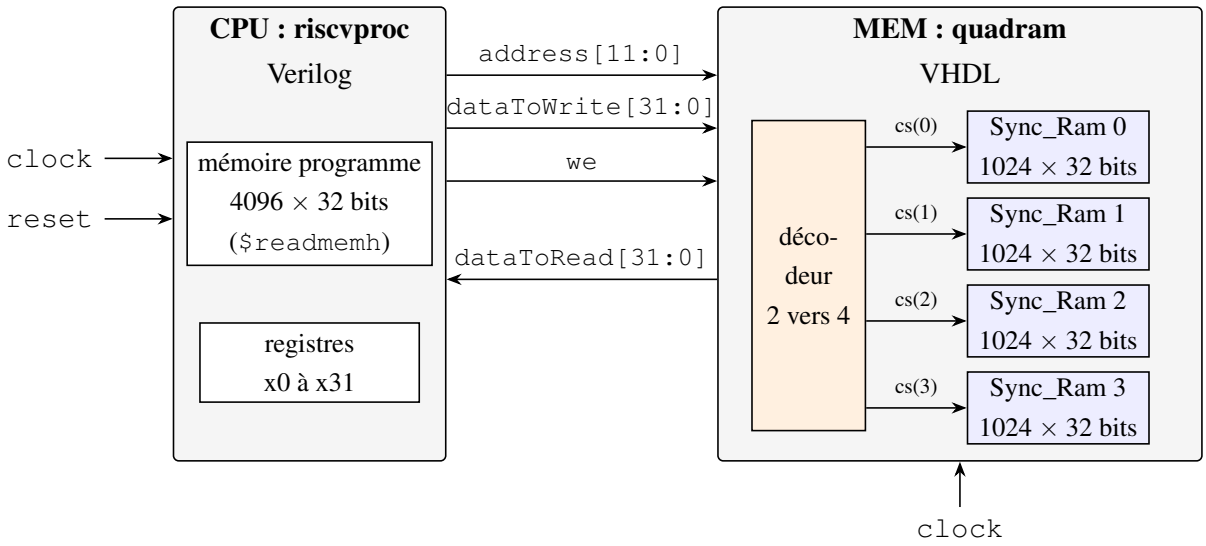



FIGURE 2 – Système final : processeur et bloc `quadram`. Les bits 11–10 de l'adresse attaquent le décodeur ; les bits 9–0, la donnée à écrire, `we` et l'horloge sont communs aux quatre RAM, dont les sorties sont reliées (bus trois états).

## 5.1 Accès aux données dans la mémoire externe

Les données n'étant plus dans `memory`, seules les instructions de manipulation des données changent : `lw` et `sw` passent maintenant par le bus (code 8). `sw` place l'adresse, la donnée et `we` = 1 sur le bus ; `lw`

place l'adresse, puis recopie `dataToRead` dans le registre destination. Les autres instructions ne sont pas modifiées.

Code 8 – Instructions `lw` et `sw` reliées au bloc `quadram`

```
`LW: begin
  address    = regs[`RS1] + `I_IMM;    // address on the bus (falling edge)
  @(negedge clock);                    // rising edge in between: the quadram samples
   the address
  regs[`RD] = dataToRead;              // the word read is valid now
end
...
`SW: begin
  address     = regs[`RS1] + `S_IMM; // address and data on the bus
  dataToWrite = regs[`RS2];
  we          = 1;
  @(negedge clock);                    // rising edge in between: the quadram writes
   the word
  we          = 0;
end
```

## 5.2 Synchronisation avec le bloc quadram

Le bloc `quadram` est synchrone : l'écriture a lieu sur le front montant de l'horloge, et la lecture passe par le registre d'adresse `read_address` de `Sync_Ram` : la donnée lue ne correspond à une nouvelle adresse qu'après le front montant qui a mémorisé cette adresse. Le processeur travaillant sur le front descendant, un accès se déroule ainsi :

1. front descendant : le processeur place l'adresse sur le bus et, pour `sw`, la donnée et `we` = 1 ;
2. front montant suivant : la RAM sélectionnée écrit la donnée, ou mémorise l'adresse de lecture ;
3. front descendant suivant : `lw` recopie `dataToRead`, valide depuis le front montant ; `sw` remet `we` à 0.

L'attente est réalisée par un second `@(negedge clock)` dans l'instruction : `lw` et `sw` durent deux périodes, les autres instructions une seule. Le tableau 10 et la figure 3 montrent cette séquence pour `test_base` exécuté sur le système. À 140 ns, `lw x9, 4(x2)` place l'adresse 104, mais `dataToRead` vaut encore $-7$, le mot 96 écrit juste avant ; la donnée 107 n'apparaît qu'après le front montant de 145 ns. Sans l'attente, `lw` lirait donc l'ancien mot.

| Temps | Front | Instruction | Bus et registres |
|---|---|---|---|
| 100 ns | ↓ | `sw x3, 4(x2)` | `address` = 104, `dataToWrite` = 107, `we` = 1 |
| 105 ns | ↑ | | la RAM 0 écrit 107 au mot 104 |
| 110 ns | ↓ | | `we` = 0 |
| 120 ns | ↓ | `sw x1, -4(x2)` | `address` = 96, `dataToWrite` = −7, `we` = 1 |
| 140 ns | ↓ | `lw x9, 4(x2)` | `address` = 104, `dataToRead` = −7 (mot 96) |
| 145 ns | ↑ | | adresse 104 mémorisée : `dataToRead` = 107 |
| 150 ns | ↓ | | x9 = 107 |
| 160 ns | ↓ | `lw x10, -4(x2)` | instruction suivante |

TABLE 10 – Accès à la mémoire pendant `test_base` sur le système

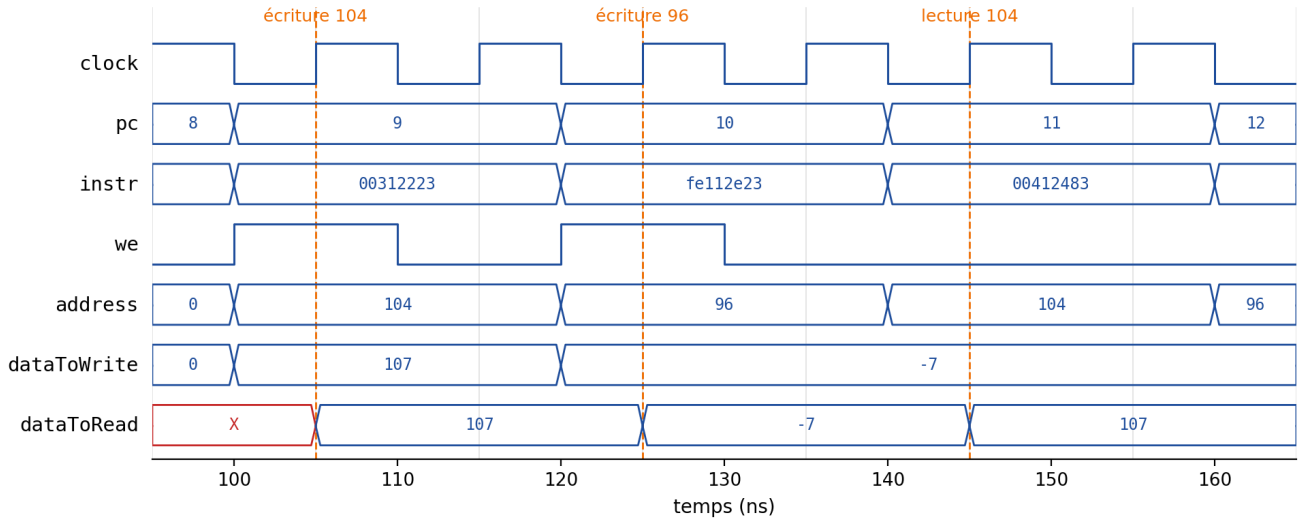



FIGURE 3 – Écritures puis lecture dans le bloc `quadram` : la donnée lue n'est valide qu'après le front montant

## 5.3 Micro-architecture du processeur

Les ports, l'horloge et le reset nécessaires à cet interfaçage ont été ajoutés à la section 4.4. Comme le processeur travaille sur le front descendant et la mémoire sur le front montant, aucune autre modification n'a été nécessaire.

## 5.4 Bloc système en VHDL

Le fichier `systeme.vhd` (code 9) contient :

— la déclaration du `component riscvproc`, avec le même nom et les mêmes ports que le module Verilog, écrits avec des types VHDL (`std_logic` et `std_logic_vector`) ;

— les signaux internes qui forment le bus entre les deux blocs ;
— l'instance du bloc `quadram` générique de la partie 3, avec `Nbits_sel` = 2, `Nbits_addr` = 10 et `Nbits_data` = 32 : quatre RAM de 1024 mots de 32 bits, soit 4096 mots adressés sur 12 bits, comme le bus d'adresse du processeur ;
— les conversions de type : les ports de `quadram` sont de type `unsigned`, ceux du processeur de type `std_logic_vector`.

Code 9 – Extrait de `systeme.vhd`

```
component riscvproc is
  port (
    clock       : in  std_logic;
    reset       : in  std_logic;
    address     : out std_logic_vector(11 downto 0);
    dataToWrite : out std_logic_vector(31 downto 0);
    dataToRead  : in  std_logic_vector(31 downto 0);
    we          : out std_logic
  );
end component riscvproc;
...
CPU: riscvproc
  port map (clock => clock, reset => reset, address => address,
            dataToWrite => dataToWrite, dataToRead => dataToRead, we => we);

-- 4 RAMs of 1024 words of 32 bits: address(11 downto 10) selects the RAM,
-- address(9 downto 0) is the word inside the RAM
MEM: entity work.quadram(RTL)
  generic map (Nbits_sel => 2, Nbits_addr => 10, Nbits_data => 32)
  port map (clock => clock, we => we,
            address => unsigned(address),
            datain  => unsigned(dataToWrite),
            dataout => dataOut);

-- the quadram works with unsigned, the processor with std_logic_vector
dataToRead <= std_logic_vector(dataOut);
```

Le testbench `systeme_tb.vhd` génère l'horloge (période 10 ns) et le reset (à 1 jusqu'à 12 ns) ; un processus de surveillance arrête la simulation au bout de 100 $\mu$s si le programme n'atteint jamais `ebreak`. Les blocs des parties 2 et 3 sont réutilisés sans copie : ils sont compilés depuis leurs dossiers, dans la même bibliothèque `work` que le processeur (code 10).

Code 10 – Compilation et simulation du système dans ModelSim

```
vlib work
vlog -sv riscvproc.sv
vcom -93 ../p2_q5/ramchip.vhd ../p3_q4/decoder.vhd ../p3_q4/quadram_for.vhd systeme.
```

```
    vhd systeme_tb.vhd
vsim -voptargs=+acc -onfinish stop work.systeme_tb
run -all
```

ModelSim associe le `component` au module Verilog de même nom présent dans la bibliothèque. Si ce module n'a pas été compilé, ModelSim ne donne qu'un avertissement (« Component instance "CPU : riscvproc" is not bound ») et simule le système sans processeur : c'est ce qui nous est arrivé au premier essai, et la simulation s'est alors arrêtée sur le délai de surveillance de 100 $\mu$s.

## 5.5 Validation

**Programme instr2.** À la fin du programme, x1 = 1 et x2 = 6, et le mot 65 se trouve maintenant dans la RAM 0 du bloc `quadram` (`RAMS(0)/RAM/ram(65)` = 6, figure 4). La simulation se termine à 80 ns au lieu de 70 ns, car l'instruction `sw` dure deux périodes.

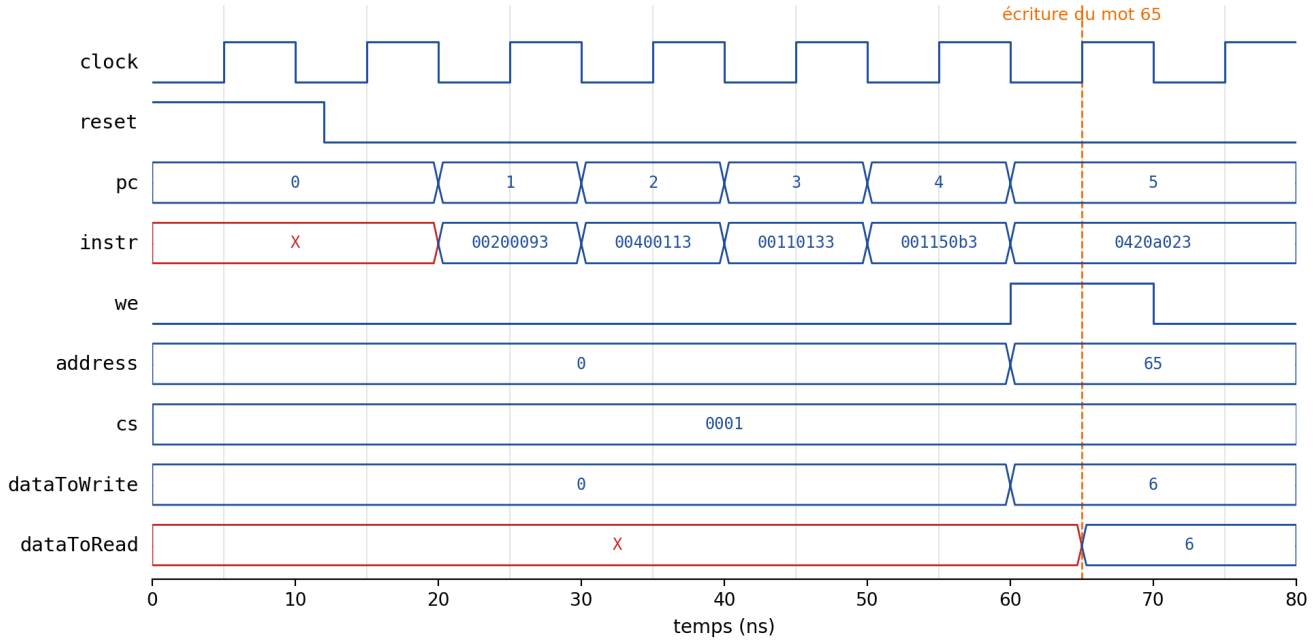



FIGURE 4 – Système complet avec le programme `instr2`

**Accès aux quatre RAM.** Le programme `test_memoire` écrit 11, 22, 33 et 44 au même déplacement 5 dans chacune des quatre RAM (mots 5, 1029, 2053 et 3077), puis relit ces mots ; il teste aussi un déplacement négatif, la frontière entre les RAM 0 et 1 (mot 1023) et une lecture juste après une écriture à la même adresse. Les résultats sont ceux attendus (tableau 11). Pendant les quatre écritures, le signal `cs` du décodeur prend successivement les valeurs `0001`, `0010`, `0100` et `1000` (figure 5). La simulation se termine à 350 ns : le programme compte 23 instructions, dont 11 `lw` ou `sw` de deux périodes, soit 34 périodes à partir de la première instruction à 20 ns.

| Instructions | Destination | Résultat |
|---|---|---|
| 4 `sw` au déplacement 5 de chaque RAM | mots 5, 1029, 2053, 3077 | 11, 22, 33, 44 |
| 4 `lw` de ces mots | x20, x21, x22, x23 | 11, 22, 33, 44 |
| `add` des quatre valeurs | x24 | 110 |
| `lw x25, -5(x8)`, avec x8 = 3082 | x25 | 44 |
| `sw x24, -1(x5)`, avec x5 = 1024 | mot 1023 | 110 |
| `lw x26, 1023(x0)` | x26 | 110 |

TABLE 11 – Résultats de `test_memoire` sur le système

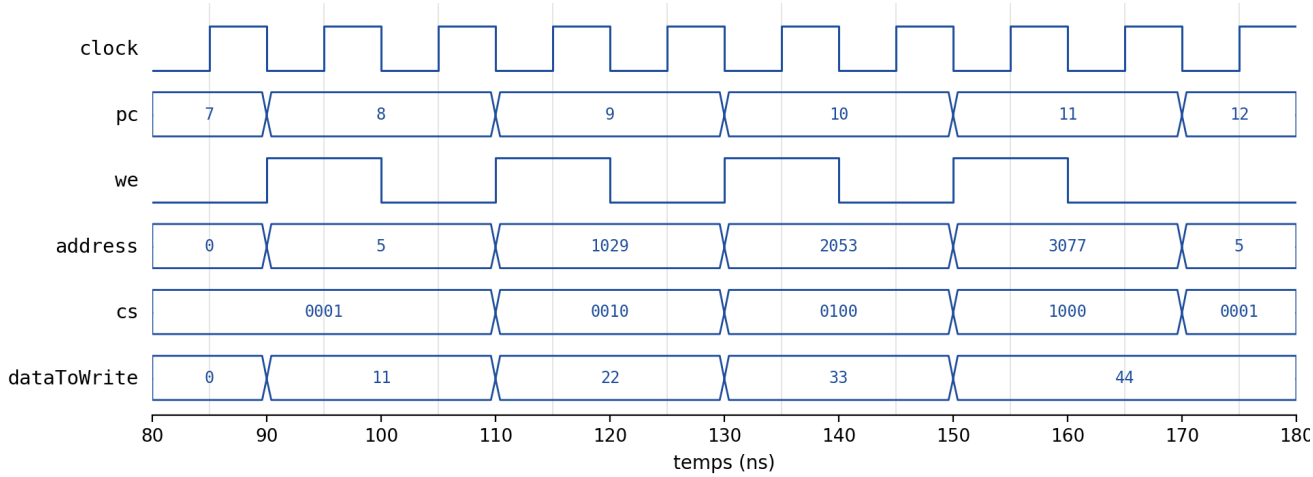



FIGURE 5 – Les quatre écritures de `test_memoire` : le décodeur sélectionne successivement les quatre RAM

Ce test a révélé une erreur dans notre `Sync_Ram` de la partie 2 : l'écriture n'était pas conditionnée par `cs` (`if we = '1' then`). Comme les quatre RAM partagent l'adresse, la donnée et `we`, chaque `sw` écrivait dans les quatre RAM au même déplacement, et les quatre mots relus valaient tous 44 (x24 = 176 au lieu de 110). La condition est devenue `if we = '1' and cs = '1' then`. Le testbench de `quadram` de la partie 3 détecte aussi cette erreur (messages d'erreur sur les RAM 0 à 2) lorsqu'il est exécuté avec la RAM non corrigée.

**Programme test_base.** Les registres ont les mêmes valeurs qu'à la section 4.4, et les mots 104 et 96 se trouvent maintenant dans la RAM 0 du bloc `quadram` (107 et −7). La simulation se termine à 200 ns au lieu de 160 ns : les quatre accès à la mémoire durent chacun une période de plus.

### 5.6 Différences avec le premier programme

**À compléter :** comparer `instr2.txt` avec le premier programme de test (`instr.txt`, sur Madoc) : décoder les deux programmes instruction par instruction, en particulier les adresses des `lw` et `sw`, et expliquer pourquoi le premier programme devait être modifié pour fonctionner avec le système.

## Conclusion

**À compléter :** bilan du travail réalisé, difficultés rencontrées et perspectives (parties 6 et 7 : synthèse sur FPGA, compatibilité avec un flot de compilation RISC-V).

# Table des figures

1 Processeur avec horloge et reset, début de `test_rv32im` : une instruction par période . . 12
2 Système final : processeur et bloc `quadram`. Les bits 11–10 de l'adresse attaquent le décodeur ; les bits 9–0, la donnée à écrire, `we` et l'horloge sont communs aux quatre RAM, dont les sorties sont reliées (bus trois états). . . . . 12
3 Écritures puis lecture dans le bloc `quadram` : la donnée lue n'est valide qu'après le front montant . . . . . 14
4 Système complet avec le programme `instr2` . . . . . 16
5 Les quatre écritures de `test_memoire` : le décodeur sélectionne successivement les quatre RAM . . . . . 17

# Liste des tableaux

1 Programme `instr2.txt` . . . . . 4
2 Formats d'instructions utilisés (spécification RISC-V, chapitre 2.2) . . . . . 4
3 Codage de `add x9, x20, x21` . . . . . 5
4 Modes d'adressage . . . . . 5
5 Résultats de `test_base` (x2 = 100, x6 = 3 et x7 = 800 sont corrects dans les deux cas) . 6
6 Instructions de l'extension M (opcode `0110011`, funct7 = `0000001`) . . . . . 8
7 Résultats imposés par la spécification pour la division . . . . . 8
8 Résultats de `test_rv32im`, avec x1 = $-7$, x2 = 2, x11 = $-2^{31}$ et x12 = $-1$ . . . . . 10
9 Ports ajoutés au processeur . . . . . 10
10 Accès à la mémoire pendant `test_base` sur le système . . . . . 14
11 Résultats de `test_memoire` sur le système . . . . . 17

# Bibliographie

[1] Sébastien PILLEMENT. *Mini-projet : Réutilisation de codes HDL et HDL réutilisable*. Sujet de mini-projet, ETN 5, Polytech Nantes. 2026.

[2] Andrew WATERMAN et Krste ASANOVIĆ. *The RISC-V Instruction Set Manual, Volume I : Unprivileged ISA*. Document Version 20191213. RISC-V Foundation. 2019. URL : https://riscv.org/technical/specifications/.